1.FUNCTIONAL REQUIREMENT

1. The system shall accept and parse valid OPENAPI 3.0
   YAML<L/JSON specifications Provided by the user

2. The system shall generate dynamic HTTP mock endpoints
   Based on the API paths and operations defined in the OPENAPI specification.

3. The system shall generate randomized JSON responses that comply with the
   Schemas defined in the OPENAPI specification.

4. The system shall validate incoming requests against the request schemas defined
   In the OPENAPI specifications
5. The system shall allow users to config simulated response latency for generated
   Mock endpoints.

2.NON-FUNCTIONAL REQUIREMENT

- The mock server engine shall sustain at least 1,000 mock API requests per second

  While maintaining the configured response latency within +- 10ms

- The system Shall validate API specifications and incoming requests securely and shall Prevent invalid or unauthorized from causing unintended server behaviour.

REQ ID	TYPE	DESCRIPTION	PRIORITY	ACCEPTANCE CRITERIA	RATIONALE (SHORT)
FR:001	FUNCTIONAL	The system shall accept and parse valid OPENAPI 3.0 Provided by the user	HIGH	Valid OPENAPI specification is successfully parsed	Parsing the specification is the starting point for generating mock APIs.
FR:002	FUNCTIONAL	The system shall generate **dynamic HTTP mock endpoints** based on the API paths and operations defined in the OPENAPI specification.	HIGH	Every valid API path generates a corresponding mock endpoint.	Developers and QA engineers need working mock endpoints for API testing.
FR:003	FUNTIONAL	The system shall generate **randomized JSON responses that comply with the schemas** defined in the OPENAPI specification.	HIGH	Valid requests are processed and invalid requests are rejected.	Request validation helps QA engineers test API behavior and input validation.
FR:004	FUNCTIONAL	The system shall validate incoming requests against the **request schemas** defined in the OpenAPI specification.	HIGH	Valid requests are processed and invalid requests are rejected.	Request validation helps QA engineers test API behaviour and input validation.
FR:005	FUNCTIONAL	The system shall allow users to **configure simulated response latency** for generated mock endpoints.	MEDIUM	The response delay matches the user-configured latency within the specified tolerance.	Simulated latency helps test application behaviour under different network/API response conditions.
NFR-001	Performance	The mock server engine shall sustain **at least 1,000 mock API requests per second** while maintaining the configured response latency within **±10 ms**.	HIGH	Benchmarking confirms ≥1,000 requests/sec and latency accuracy within ±10 ms under peak load	The problem statement specifically requires the mock server to handle high request throughput.
NFR-002	Security	The system shall validate API specifications and incoming requests securely and shall prevent **invalid or unauthorized requests from causing unintended server behavior**.	High	Security testing shows invalid/malicious inputs are rejected without compromising the mock server	Since the tool exposes HTTP endpoints, secure input handling is necessary to protect the mock server.

3.ACTORS & USECASES

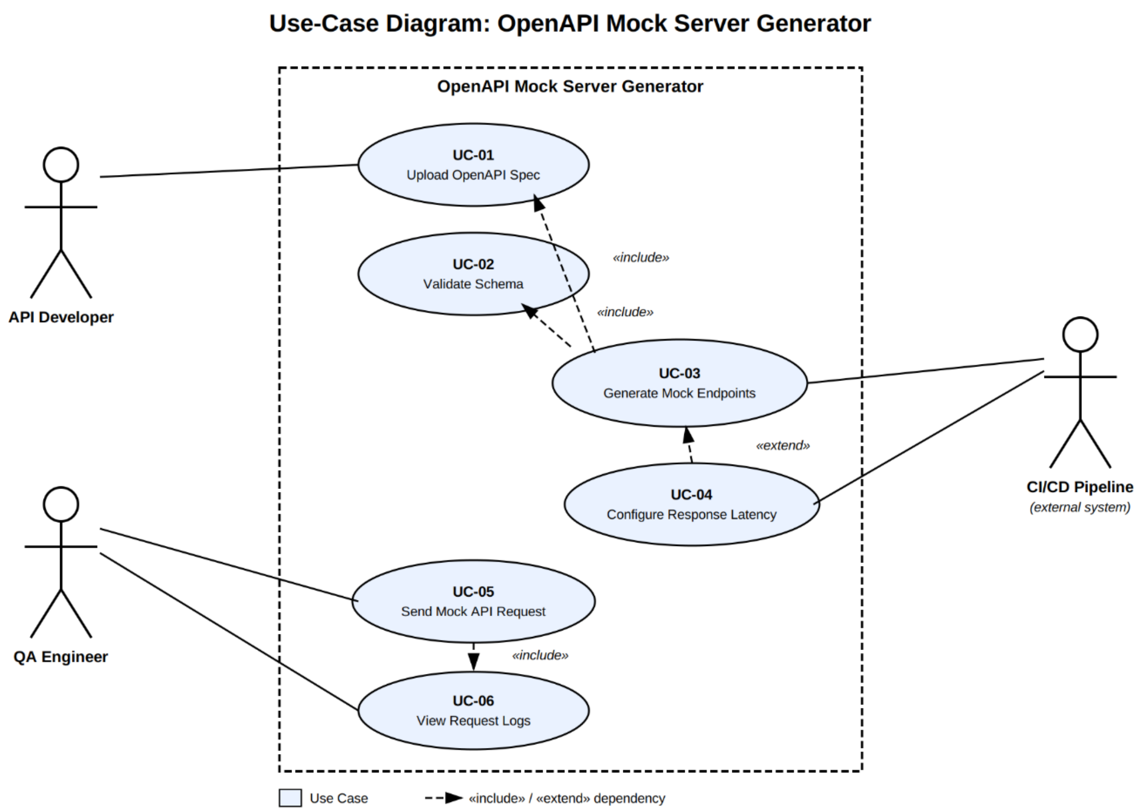
API Developer : Provides the OpenAPI 3.0 YAML/JSON specification and configures the mock server.

QA Engineer: Uses the generated mock APIs to test API requests, responses, validation, and latency behavior.

API Client: Sends HTTP requests to the generated mock REST endpoints and receives simulated responses.

ID	Use Case	Main Actor
UC01	Upload/Provide OpenAPI Specification	API Developer
UC02	Parse OpenAPI Specification	API Developer
UC03	Generate Mock REST Endpoints	API Developer
UC04	Validate API Request	QA Engineer / API Client
UC05	Generate Schema-Compliant Response	QA Engineer / API Client
UC06	Configure Response Latency	API Developer
UC07	Test Mock API Endpoint	QA Engineer / API Client

4.UML CASE DiAGRAM.



USE-CASE FLOW SPECIFICATION.

Preconditions:

1. API Developer has an OpenAPI 3.0 YAML/JSON specification.
2. The specification is available to the system.
3. The Mock Server Generator is ready to process the specification.

Postconditions:

1. Mock REST endpoints have been generated successfully.
2. The endpoints are available for testing.
3. Requests can be validated against their schemas.
4. Schema-compliant JSON responses can be generated.
5. Configured response latency is applied.

MAIN SUCCESS SCANARIO :

1. API Developer provides an OpenAPI 3.0 YAML/JSON specification.
2. System receives the specification.
3. System parses the OpenAPI specification.
4. System validates the specification.
5. System identifies the API paths and operations.
6. System generates corresponding mock REST endpoints.
7. System associates the defined schemas with the generated endpoints.
8. System generates schema-compliant JSON responses.
9. API Developer configures simulated response latency.
10. System applies the configured latency.
11. QA Engineer sends a request to a mock endpoint.
12. System validates the request.
13. System returns the simulated JSON response.
14. Use case ends successfully.

ALTERNATIVE FLOW:

Main Flow

↓

Validate specification

↓

INVALID

↓

Alternate Flow

↓

Display error message

↓

Do NOT generate endpoints

↓

Developer corrects specification